CASCADE Deletion Fix - Complete Guide

Problem

When deleting a post, you get this error:

`SyntaxError: Failed to execute 'json' on 'Response': Unexpected end of JSON input`

Root Cause: Foreign key constraints prevent deletion when comments or tags exist.

Solution: Add CASCADE to Foreign Keys

Part 1: Update Database Schema

File: `backend/schema.sql`

```
-- Drop existing tables (in correct order)
DROP TABLE IF EXISTS post_tags;
DROP TABLE IF EXISTS tags;
DROP TABLE IF EXISTS comments;
DROP TABLE IF EXISTS images;
DROP TABLE IF EXISTS posts;
DROP TABLE IF EXISTS users;

-- Users table
CREATE TABLE users (
    id TEXT PRIMARY KEY,
    email TEXT UNIQUE NOT NULL,
    username TEXT UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    full_name TEXT,
    bio TEXT,
    avatar_url TEXT,
    role TEXT DEFAULT 'user',
    created_at INTEGER NOT NULL,
    updated_at INTEGER NOT NULL
);

-- Posts table with CASCADE on author
CREATE TABLE posts (
    id TEXT PRIMARY KEY,
    title TEXT NOT NULL,
    slug TEXT UNIQUE NOT NULL,
    content TEXT NOT NULL,
    excerpt TEXT,
    author_id TEXT NOT NULL,
```

```
    status TEXT DEFAULT 'draft',
    featured_image TEXT,
    published_at INTEGER,
    created_at INTEGER NOT NULL,
    updated_at INTEGER NOT NULL,
    views INTEGER DEFAULT 0,
    FOREIGN KEY (author_id) REFERENCES users(id) ON DELETE CASCADE
);

-- Tags table
CREATE TABLE tags (
    id TEXT PRIMARY KEY,
    name TEXT UNIQUE NOT NULL,
    slug TEXT UNIQUE NOT NULL,
    created_at INTEGER NOT NULL
);

-- Post-Tags junction table with CASCADE
CREATE TABLE post_tags (
    post_id TEXT NOT NULL,
    tag_id TEXT NOT NULL,
    PRIMARY KEY (post_id, tag_id),
    FOREIGN KEY (post_id) REFERENCES posts(id) ON DELETE CASCADE,
    FOREIGN KEY (tag_id) REFERENCES tags(id) ON DELETE CASCADE
);

-- Comments table with CASCADE DELETE
CREATE TABLE comments (
    id TEXT PRIMARY KEY,
    post_id TEXT NOT NULL,
    author_name TEXT NOT NULL,
    author_email TEXT NOT NULL,
    content TEXT NOT NULL,
    is_approved INTEGER DEFAULT 0,
    created_at INTEGER NOT NULL,
    FOREIGN KEY (post_id) REFERENCES posts(id) ON DELETE CASCADE
);

-- Images table (no CASCADE - images can exist independently)
CREATE TABLE images (
    id TEXT PRIMARY KEY,
    filename TEXT NOT NULL,
    size INTEGER NOT NULL,
    mime_type TEXT NOT NULL,
    variant TEXT NOT NULL,
    data BLOB NOT NULL,
```

```
    created_at INTEGER NOT NULL
);

-- Create indexes for better performance
CREATE INDEX idx_posts_author ON posts(author_id);
CREATE INDEX idx_posts_status ON posts(status);
CREATE INDEX idx_posts_slug ON posts(slug);
CREATE INDEX idx_comments_post ON comments(post_id);
CREATE INDEX idx_post_tags_post ON post_tags(post_id);
CREATE INDEX idx_post_tags_tag ON post_tags(tag_id);
```

Part 2: Apply Schema to Database

Option A: Fresh Database

```
# Connect to Turso
turso db shell blog-production

# Drop all tables
DROP TABLE IF EXISTS post_tags;
DROP TABLE IF EXISTS tags;
DROP TABLE IF EXISTS comments;
DROP TABLE IF EXISTS images;
DROP TABLE IF EXISTS posts;
DROP TABLE IF EXISTS users;

# Copy and paste the CREATE TABLE statements from schema.sql above

# Verify CASCADE is set
.schema comments
-- Should show: FOREIGN KEY (post_id) REFERENCES posts(id) ON DELETE CASCADE

.exit
```

Option B: Using Schema File

```
# Save the schema to backend/schema.sql
cd backend

# Apply to database
turso db shell blog-production < schema.sql

# Verify
turso db shell blog-production
.schema comments
```

```
.exit
```

Option C: Keep Data (Advanced)

```
# 1. Backup existing data
turso db shell blog-production ".dump" > backup.sql

# 2. Drop and recreate tables with CASCADE
turso db shell blog-production

DROP TABLE IF EXISTS post_tags;
DROP TABLE IF EXISTS comments;

-- Recreate with CASCADE
CREATE TABLE comments (
    id TEXT PRIMARY KEY,
    post_id TEXT NOT NULL,
    author_name TEXT NOT NULL,
    author_email TEXT NOT NULL,
    content TEXT NOT NULL,
    is_approved INTEGER DEFAULT 0,
    created_at INTEGER NOT NULL,
    FOREIGN KEY (post_id) REFERENCES posts(id) ON DELETE CASCADE
);

CREATE TABLE post_tags (
    post_id TEXT NOT NULL,
    tag_id TEXT NOT NULL,
    PRIMARY KEY (post_id, tag_id),
    FOREIGN KEY (post_id) REFERENCES posts(id) ON DELETE CASCADE,
    FOREIGN KEY (tag_id) REFERENCES tags(id) ON DELETE CASCADE
);

.exit

# 3. Restore data from backup (if needed)
# Edit backup.sql to only include INSERT statements for comments and post_tags
turso db shell blog-production < backup-data-only.sql
```

Part 3: Update Backend Delete Handler

File: `backend/src/handlers.rs`

Find and update the `delete_post` function:

```rust
// DELETE /api/posts/:id - Delete post
pub async fn delete_post(
    State(state): State<Arc<AppState>>,
    Path(id): Path<String>,
) -> impl IntoResponse {
    match state.post_repo.delete(&id).await {
        Ok(()) => (
            StatusCode::OK,
            Json(ApiResponse::success("Post deleted successfully")),
        ),
        Err(e) => {
            eprintln!("Error deleting post: {}", e);
            (
                StatusCode::INTERNAL_SERVER_ERROR,
                Json(ApiResponse::<()>::error(e)),
            )
        }
    }
}
```

File: `backend/src/repository.rs`

Update the `delete` method:

```rust
pub async fn delete(&self, id: &str) -> Result<(), String> {
    let conn = self.pool.connection().await.map_err(|e| e.to_string())?;

    // With CASCADE, this will automatically delete:
    // - Associated comments (via post_id FK)
    // - Associated post_tags entries (via post_id FK)
    conn.execute(
        "DELETE FROM posts WHERE id = ?",
        libsql::params![id]
    )
    .await
    .map_err(|e| {
        eprintln!("Delete error: {}", e);
        e.to_string()
    })?;

    Ok(())
}
```

Part 4: Update Frontend Delete Component

File: `frontend/components/DeletePostButton.tsx`

Update the `handleDelete` function to handle empty responses:

```
const handleDelete = async () => {
  setIsDeleting(true)

  try {
    const token = localStorage.getItem('auth_token')
    if (!token) {
      alert('Not authenticated')
      router.push('/login')
      return
    }

    const API_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:3001'
    const response = await fetch(`${API_URL}/api/posts/${postId}`, {
      method: 'DELETE',
      headers: {
        'Authorization': `Bearer ${token}`,
      },
    })

    if (!response.ok) {
      throw new Error(`HTTP ${response.status}`)
    }

    // Handle potentially empty or non-JSON response
    const text = await response.text()
    let data

    if (text) {
      try {
        data = JSON.parse(text)
      } catch (e) {
        console.warn('Response is not JSON:', text)
        data = { success: true }
      }
    } else {
      // Empty response - treat as success if status is 200
      data = { success: true }
    }

    if (data.success || response.status === 200) {
      // Success!
      if (onDeleted) {
        onDeleted()
      } else {
```

```
        router.refresh()
      }

      setShowConfirm(false)
    } else {
      throw new Error(data.error || 'Delete failed')
    }
  } catch (error) {
    console.error('Failed to delete post:', error)
    alert('Failed to delete post: ' + error)
  } finally {
    setIsDeleting(false)
  }
}
```

Complete Implementation Steps

Step 1: Backup (Optional but Recommended)

```
turso db shell blog-production

.dump > backup_$(date +%Y%m%d).sql

.exit
```

Step 2: Apply CASCADE Schema

```
# Connect to database
turso db shell blog-production

# Drop existing tables with FK constraints
DROP TABLE IF EXISTS post_tags;
DROP TABLE IF EXISTS comments;

# Create with CASCADE
CREATE TABLE comments (
    id TEXT PRIMARY KEY,
    post_id TEXT NOT NULL,
    author_name TEXT NOT NULL,
    author_email TEXT NOT NULL,
    content TEXT NOT NULL,
    is_approved INTEGER DEFAULT 0,
    created_at INTEGER NOT NULL,
    FOREIGN KEY (post_id) REFERENCES posts(id) ON DELETE CASCADE
);
```

```
CREATE TABLE post_tags (
    post_id TEXT NOT NULL,
    tag_id TEXT NOT NULL,
    PRIMARY KEY (post_id, tag_id),
    FOREIGN KEY (post_id) REFERENCES posts(id) ON DELETE CASCADE,
    FOREIGN KEY (tag_id) REFERENCES tags(id) ON DELETE CASCADE
);

# Verify CASCADE
.schema comments
-- Look for: ON DELETE CASCADE

.exit
```

Step 3: Update Backend Code

Update `backend/src/repository.rs` with the delete method shown above.

Step 4: Update Frontend Code

Update `frontend/components/DeletePostButton.tsx` with the handleDelete shown above.

Step 5: Rebuild and Test

```
# Backend
cd backend
cargo clean
cargo build
cargo run

# Frontend (new terminal)
cd frontend
npm run dev

# Test deletion
# 1. Create a test post
# 2. Add a comment to it
# 3. Try deleting the post
# 4. Should work without errors!
```

Verify CASCADE is Working

```
turso db shell blog-production

-- Check the schema includes CASCADE
.schema comments
-- Should show: FOREIGN KEY (post_id) REFERENCES posts(id) ON DELETE CASCADE

-- Test CASCADE behavior
SELECT COUNT(*) FROM posts;
SELECT COUNT(*) FROM comments;

-- Create test data
INSERT INTO posts (id, title, slug, content, author_id, created_at, updated_at)
VALUES ('test-post', 'Test', 'test', 'Content', 'admin-id', strftime('%s', 'now'), strftime

INSERT INTO comments (id, post_id, author_name, author_email, content, created_at)
VALUES ('test-comment', 'test-post', 'Test', 'test@test.com', 'Test comment', strftime('%s'

-- Verify comment exists
SELECT * FROM comments WHERE post_id = 'test-post';

-- Delete the post
DELETE FROM posts WHERE id = 'test-post';

-- Verify comment was automatically deleted
SELECT * FROM comments WHERE post_id = 'test-post';
-- Should return no rows (CASCADE worked!)

.exit
```

What Gets Deleted (CASCADE Behavior)

When you delete a post with `DELETE FROM posts WHERE id = 'post-id'`:

Table	What Happens	Why
comments	Auto-deleted	`ON DELETE CASCADE` on `post_id` FK
post_tags	Auto-deleted	`ON DELETE CASCADE` on `post_id` FK

Table	What Happens	Why
images	Kept	No FK to posts (images can be reused)
users	Kept	Post author remains
tags	Kept	Tags can be reused

Troubleshooting

Error: "FOREIGN KEY constraint failed"

Cause: CASCADE not set properly

Fix:

```
turso db shell blog-production
.schema comments
# If you don't see "ON DELETE CASCADE", recreate the table
```

Error: "database is locked"

Cause: Multiple connections

Fix:

```
# Close all Turso shells
# Wait 10 seconds
# Try again
```

Error: "table already exists"

Fix:

```
turso db shell blog-production
DROP TABLE IF EXISTS comments;
-- Then recreate with CASCADE
```

Deletion Still Fails

Debug:

```
# Check backend logs
cd backend
RUST_LOG=debug cargo run
```

```
# Try deleting a post
# Watch for SQL errors in console
```

Common issues: 1. Token expired - logout and login again 2. Wrong post ID 3. Backend not running 4. CASCADE not applied to database

---

Alternative: Manual CASCADE (If Can't Recreate Tables)

If you absolutely cannot recreate tables, implement manual CASCADE in code:

File: `backend/src/repository.rs`

```rust
pub async fn delete(&self, id: &str) -> Result<(), String> {
    let conn = self.pool.connection().await.map_err(|e| e.to_string())?;

    // Manually delete in correct order

    // 1. Delete comments first
    conn.execute(
        "DELETE FROM comments WHERE post_id = ?",
        libsql::params![id]
    )
    .await
    .map_err(|e| format!("Failed to delete comments: {}", e))?;

    // 2. Delete post-tag relationships
    conn.execute(
        "DELETE FROM post_tags WHERE post_id = ?",
        libsql::params![id]
    )
    .await
    .map_err(|e| format!("Failed to delete post tags: {}", e))?;

    // 3. Finally delete the post
    conn.execute(
        "DELETE FROM posts WHERE id = ?",
        libsql::params![id]
    )
    .await
    .map_err(|e| format!("Failed to delete post: {}", e))?;

    Ok(())
}
```

Note: Database CASCADE is better because: - More reliable - Database enforced - Works even with direct SQL queries - Standard SQL feature

---

Success Checklist

After implementing:

- ☐ CASCADE added to schema
- ☐ Schema applied to Turso database
- ☐ Verified with `.schema comments`
- ☐ Backend delete method updated
- ☐ Frontend handles responses correctly
- ☐ Backend rebuilt and restarted
- ☐ Frontend rebuilt and restarted
- ☐ Created test post with comment
- ☐ Deleted test post successfully
- ☐ Verified comment auto-deleted
- ☐ No more JSON parse errors

Summary

The Problem

Foreign key constraints prevented post deletion when related data existed.

The Solution

Add `ON DELETE CASCADE` to foreign keys in: - `comments` table (`post_id` references `posts`) - `post_tags` table (`post_id` references `posts`)

The Result

Deleting a post automatically deletes: - All its comments - All its tag relationships - No manual cleanup needed - No errors!

Done!

Your blog now has proper CASCADE deletion. When you delete a post: 1. All comments are automatically deleted 2. All tag relationships are automatically deleted 3. No more foreign key errors 4. Clean, automatic cleanup

Enjoy your working delete functionality!